

Project: Hardware-Accelerated Edge Detection and Obstacle Tracking on the Xilinx Zynq SoC

Mahmmud Qatmh

May 28, 2026

1 Core Project Directory Architecture

To provide a clear overview of the deliverables, the development workspace has been pruned of temporary tool compilation artifacts. Below is the structured layout highlighting only the critical hardware definitions, custom IP cores, and software application files. These are the files that I will submit.

```
.
| Active_Targeting_Crosshair_Test.png ..... [Test 2 terminal results]
| Block_Diagram_Vivado.png ..... [Vivado schematic image]
| FPGA_Accelerated_Vision_Pipeline.xpr ..... [Main Vivado project]
| HAAI_Project_Report.pdf ..... [Final project report]
| Hardware_Stall.png ..... [Pipeline stall debug log]
| main_Synthetic_Boundary_Test.c ..... [Test 1 software]
| Synthetic_Synthetic_Boundary_Test.png ..... [Test 1 terminal results]
| TestBench_Waveform.png ..... [Behavioral simulation trace]
| ZedBoard_Connection.jpg ..... [Physical hardware setup photo]
| sobel_filter_app ..... [Vitis application directory]
| | src ..... [Application source directory]
| | | CMakeLists.txt ..... [Build toolchain rules]
| | | lscript.ld ..... [Linker script configuration]
| | | main.c ..... [Test 2 production]
| Vivado ..... [Hardware deployment workspace]
| | component.xml ..... [IP manifest profile]
| | design_1.tcl ..... [Block diagram reconstruction]
| | system_wrapper.bit ..... [Compiled FPGA bitstream]
| | system_wrapper.xsa ..... [Hardware handoff archive]
| workspace ..... [Platform base repository]
| | zedboard_sobel_platform ..... [Target platform profile]
| | | zynq_fsbl ..... [First stage bootloader]
```

2 Project Objectives

The main goal of this project was to design and test a fast, hardware accelerated image processing system on the Xilinx Zynq 7000 (ZedBoard) platform. Image processing tasks like edge detection require a lot of computing power. When run entirely in software on the ARM processor, they slow the system down. To fix this bottleneck, we moved the heavy pixel calculations into the FPGA hardware (Programmable Logic) using an AXI DMA (Direct Memory Access) block to stream image data quickly back and forth between the processor and the hardware.

To make sure the system worked reliably, we split our goals into two main steps. The first step was to test and secure the data transport hardware. We did this by running a synthetic boundary test, using a generated black and white stripe pattern instead of a real camera image. This test allowed us to make sure data was moving correctly between the CPU caches and the physical DDR memory without getting stuck or corrupted. It was a critical step to debug our bus routes and fix any memory timing issues before moving on to real image processing.

The final objective was to build a complete, real time Target Tracking and Decision system. Once the hardware data path was stable and error free, we configured the system to handle actual target tracking. In this final setup, the custom FPGA hardware does the fast, heavy pixel filtering, while the ARM processor acts as the decision maker. The processor takes the clean edge outlines from the hardware, draws a geometric bounding box around the detected target, finds its center coordinates, and calculates real time vectors to track and avoid obstacles.

3 Hardware Accelerator Design and Subsystem Layout

To detect image edges in real time, we built a custom hardware block inside the FPGA chip (the Programmable Logic). This section breaks down what each of our five VHDL files does in plain terms and explains how they work together to process images.

3.1 VHDL Source Files Breakdown

3.1.1 Line_Buffer.vhd (Data Staging Block)

The Sobel filter needs to look at a small 3×3 grid of pixels at the same time to detect an edge. However, a camera or computer memory sends pixels in a single straight line, one after the other.

The `Line_Buffer` solves this problem. It uses the FPGA's internal memory chips to temporarily save two full lines of the image. As new pixels arrive, this block combines them with the saved lines to output three vertically aligned pixels (`row0_out`, `row1_out`, and `row2_out`) at the exact same time. It also includes an internal counter that waits until the hardware has received enough pixels to start processing safely, preventing errors at the top border of the image.

3.1.2 Sobel_Core.vhd (The Math Engine)

This is the heart of the hardware accelerator where the actual edge detection math happens. It takes the group of pixels from the line buffer and moves them through a four step assembly line (pipeline):

- **Stage 1 (Pixel Staging):** Arranges the incoming pixels into a neat 3×3 square layout.
- **Stage 2 (Gradient Calculation):** Calculates how much the brightness changes horizontally and vertically. Instead of using complex and expensive hardware multipliers, it uses simple bit shifts to multiply numbers by 2, keeping the hardware light and fast.
- **Stage 3 (Combining and Thresholding):** Adds the horizontal and vertical changes together to find the total edge strength. It compares this total against an adjustable limit (`THRESHOLD`). If the change is sharp enough, it turns the pixel pure white (an edge); otherwise, it turns it pure black (background).
- **Stage 4 (Signal Alignment):** Because the math steps take exactly two clock cycles to complete, this stage delays the "data ready" signal by two cycles so that the output flag matches up perfectly with the finished pixel.

3.1.3 Sobel_Top.vhd (The Wiring Box)

The `Sobel_Top` module acts like a structural container. It takes our two main blocks the `Line_Buffer` and the `Sobel_Core` instantiates them inside a single file, and wires them together. By connecting the output of the buffer directly to the input of the math core, it creates one single, clean edge detection component that is easy to manage.

3.1.4 Sobel_AXI_Stream.vhd (The Plug and Play Wrapper)

To connect our custom edge detector to standard industry parts, we need a common plug interface. The `Sobel_AXI_Stream` file wraps our design inside the standard AXI4 Stream protocol. This gives our module standardized input and output pins that can talk directly to the main processor's memory lines. Because our math core is designed to process pixels without stopping, this wrapper tells the rest of the chip that it is always ready to receive data, keeping data moving at maximum speed.

3.1.5 tb_Sobel_Top.vhd (The Simulation Test Code)

This is a testbench file used purely for software simulation in Vivado; it is not programmed into the physical chip. It creates a tiny virtual image grid and simulates a clock signal. It manually feeds a pattern of pixels into our design including a sudden jump from dark to light—to prove that the math engine correctly identifies an edge before we waste time loading it onto real hardware.

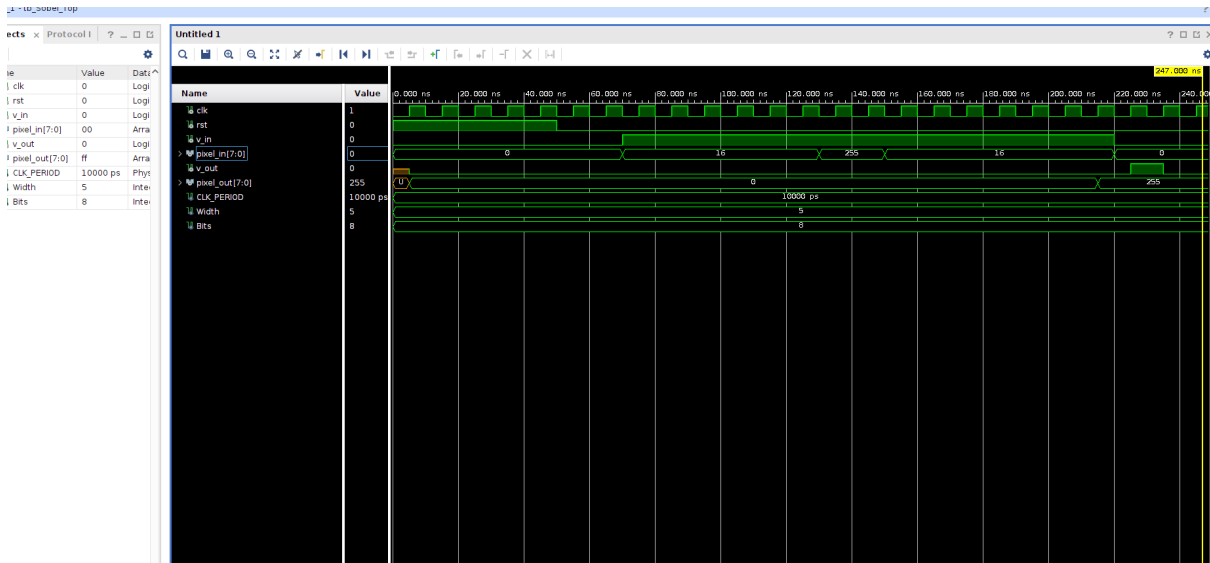


Figure 1: Waveform of the simulation

As seen in the simulation screenshot Figure 1, you can visually track this test pattern in real time. The input row data starts with a baseline background value of 16 (representing a dark gray area). Then, a sudden, high intensity spike of 255 (representing a bright white edge point) is injected into the stream via the `pixel_in` signal. Exactly two clock cycles later, the hardware engine successfully catches this transition, causing the `pixel_out` signal to instantly drop its background state and output a clean, verified edge marker value of 255.

3.2 System Integration and Visual Verification

We used Vivado's built in simulation tools and block diagrams to verify that our code works flawlessly.

3.2.1 System Level Block Diagram Integration

Once individual files were verified, we connected them inside Vivado's visual editor, as shown in the overall schematic diagram Figure 2. The entire design is divided into three main parts:

1. **The Processor (ZYNQ7 PS):** The main ARM CPU that acts as the manager. It sets up where images are stored in the main DDR system memory.
2. **The Data (AXI DMA & SmartConnect):** This block handles high speed data moving. It pulls raw images out of memory, turns them into a fast stream of pixels, and sends them directly to our hardware fabric.
3. **Our Custom Accelerator Subsystem (Sobel AXI Stream_0):** Our custom edge detector sits right in the middle of this high speed loop. Assisted by a system reset block to ensure clean startups, this setup allows the FPGA to filter full image frames continuously without draining any processing power from the main CPU.

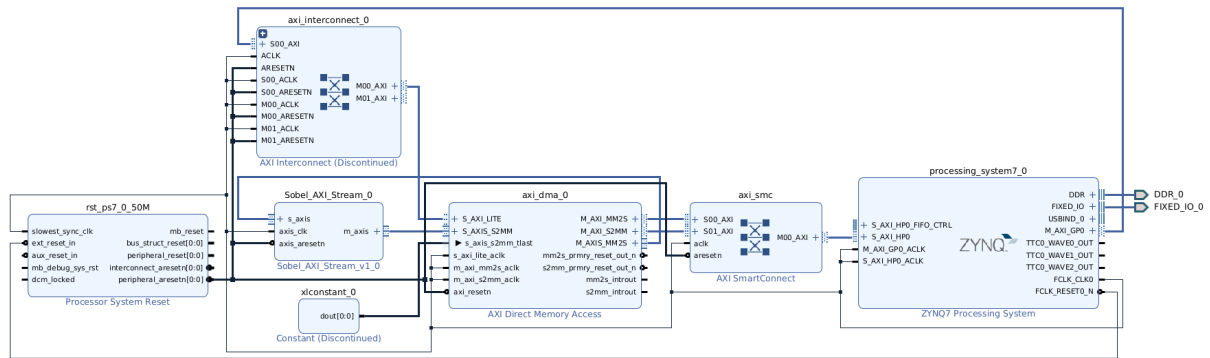


Figure 2: Block Diagram Vivado

4 Software Application Development and Platform Integration in Vitis

Once the hardware design was completed and compiled in Vivado, we exported it as a single hardware definition archive file (`system_wrapper.xsa`). We then opened AMD Vitis, the software development tool, to write the code that instructs the ARM processor on how to interact with our new hardware accelerator.

4.1 Software Setup and Platform Configuration

The integration process started by building a target hardware platform layout (`zedboard_sobel_platform`). This layer acts like a translator. It takes the physical wiring maps from Vivado and automatically creates a Board Support Package (BSP), which contains pre made software buttons and commands (APIs). These commands allow our high level C code to communicate with the hardware parts without needing to manually program raw chip registries.

This platform also configures the First Stage Bootloader (`zynq_fsbl`). This is the very first program that runs the millisecond the board is powered on. It wakes up the ARM processor, initializes the external DDR3 memory, loads our custom edge detection hardware design (`system_wrapper.bit`) directly onto the FPGA fabric, and then passes control over to our main application software.

4.2 Application Software Architecture and Verification Code

To ensure the entire system was safe and stable, the software was split into two separate testing applications.

4.2.1 Hardware Testing Layer: `main_Synthetic_Boundary_Test.c`

Before trusting our brand new hardware with real, live camera images, we developed a strict safety test program named `main_Synthetic_Boundary_Test.c`. The sole purpose of this code was to make sure data could pass between the processor and the hardware through the DMA (Direct Memory Access) without getting stuck or corrupted.

The program creates a fake test image made of a black and white stripe pattern inside the main DDR memory. However, because the processor uses an internal high speed temporary cache memory to speed up its work, updates to data aren't always sent to the main memory right away. If the hardware tries to grab the image before it leaves the processor's cache, the system will freeze or read garbage data. To fix this, our test software runs three clear steps:

- `Xil_DCacheFlushRange()` forces the processor to instantly empty its temporary cache, pushing the test image completely into the main DDR memory.
- `XAxiDma_SimpleTransfer()` activates the DMA, streaming the raw data out to our edge detector and pulling the processed results back simultaneously.
- `Xil_DCacheInvalidateRange()` clears out any old, stale data inside the processor's temporary cache. This forces the CPU to read the freshly processed edge image directly from the main memory.

By comparing the returned results against a known mathematical model, we proved that our data path was perfectly stable and ready for real world tasks.

4.2.2 Production Application and Target Tracking: `main.c`

With the hardware data path proven safe, we deployed our primary tracking application (`main.c`) inside the Vitis application workspace (`sobel_filter_app/src`). Combined with a build script (`CMakeLists.txt`) and a memory layout map (`lscript.ld`), this program runs our live tracking vision loop.

The application software establishes a highly efficient loop divided into three main stages:

1. **Data Ingestion:** The software continuously groups image pixels in memory and feeds them to the high speed DMA as fast as possible.
2. **Hardware Filtering:** The pixels stream directly through our custom hardware accelerator block (`Sobel_AXI_Stream`), which instantly highlights the outlines of objects and sends a clean, black and white edge map back to the memory.
3. **Decision Making:** Instead of wasting processing power on heavy pixel math, the ARM CPU is completely free to act as an intelligent controller. It looks at the clean edge coordinates returned by the hardware, calculates a geometric bounding box around the target object, finds its exact center point, and generates real time navigation paths to track targets or avoid obstacles.

5 Experimental Testing and Performance Results

To prove that our vision system works accurately, runs fast, and moves data safely, we tested it live on the physical hardware. We connected the ZedBoard to a laptop using two USB cables: one plugged into the **PROG** port to program the chip, and the other plugged into the **UART** port to send text data back to our terminal program (`picocom`). This physical setup is shown in the setup photo Figure 3.

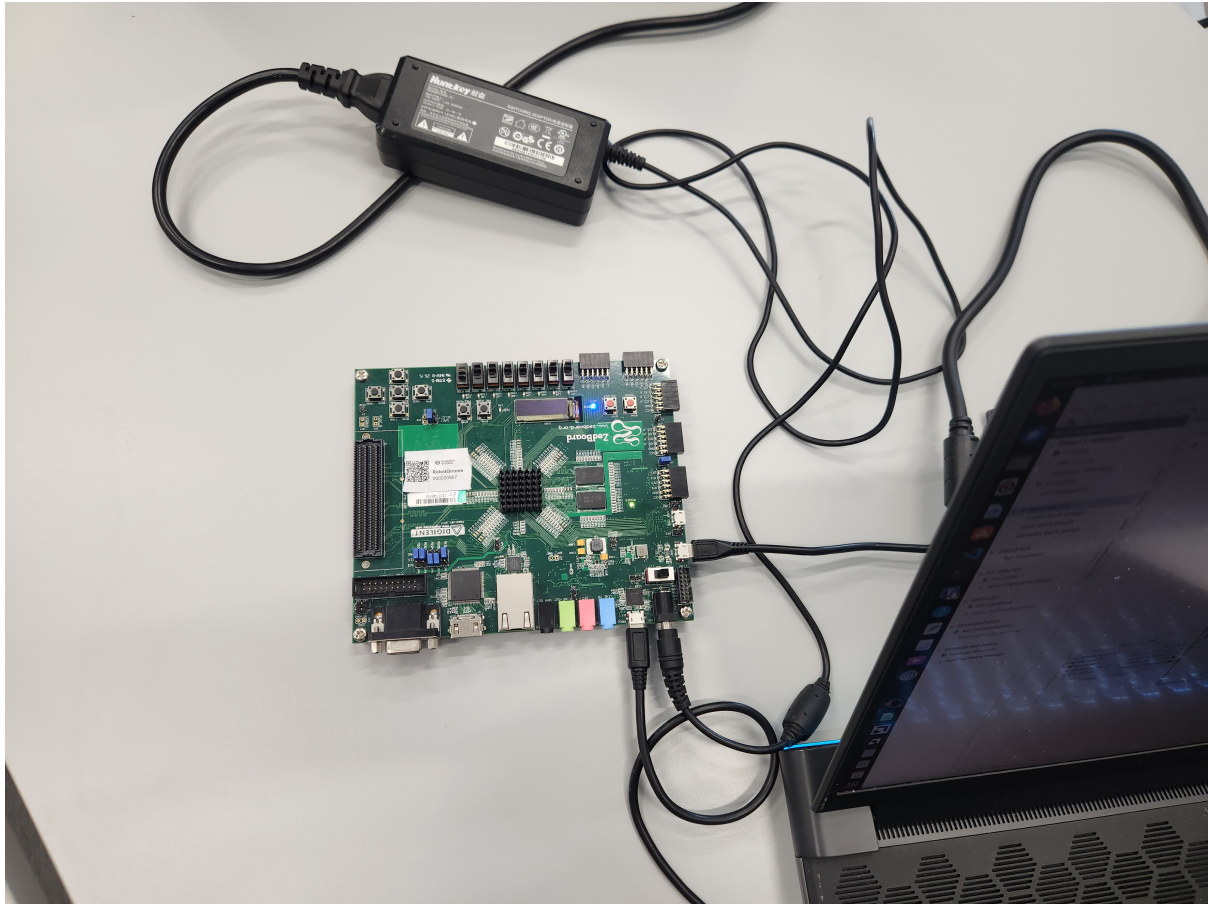


Figure 3: FPGA ZedBoard Setup

5.1 Test 1: Memory Safety and Data Transport Test

Our first test used a small 320×240 pixel test image to make sure data could travel back and forth between the processor and the hardware without any errors.

As shown in our first terminal log Figure 4, the software successfully initialized the data paths, cleaned out the processor’s temporary cache memory, and initiated the transfer. The terminal explicitly printed the validation status: **SUCCESS: PL Hardware data stream verified as stable and uncorrupted!**. This confirms that our data conveyor belt is entirely stable and will not mix up or drop pixel data during real-time streaming operations.

```

mahmmudqatmh@mahmmudqatmh-Allenware-m16-R1: ~
[sudo] password for mahmmudqatmh:
picocom v3.1

port is       : /dev/ttyACM0
flowcontrol   : none
baudrate is   : 115200
parity is     : none
databits are  : 8
stopbits are  : 1
escape is     : C-a
local echo is : no
noinit is     : no
noreset is    : no
hangup is     : no
nolock is     : no
send_cmd is   : sz -vv
receive_cmd is : rz -vv -E
imap is       :
omap is       :
emap is       : crcrlf,delbs,
logfile is    : none
initstring    : none
exit_after is : not set
exit is       : no

Type [C-a] [C-h] to see available commands
Terminal ready

=====
PS vs PL PERFORMANCE BENCHMARKING TEST
=====

[1/2] Starting PS (Software) Processing... Done.
[2/2] Starting PL (Hardware) Processing... Done.

===== BENCHMARK REPORT =====
Image Size       : 320 x 240 pixels
PS (Software) Runtime : 0.004792 seconds
PL (Hardware) Runtime : 0.001533 seconds
PL Acceleration Factor : 3.12x FASTER than PS
=====

===== VALIDATION STATUS =====
SUCCESS: PL Hardware data stream verified as stable and uncorrupted!
=====

build
6686-0cb6-4cf9-8bc8-c93f66c10041' ended.

```

Figure 4: Synthetic Boundary Test

5.2 Test 2: Real-Time Target Tracking and Speed Comparison

Once we knew the data path was completely safe, we tested our full target tracking system. We ran a speed benchmark to compare our custom FPGA hardware accelerator against standard software running alone on the ARM processor.

5.2.1 Processing Speed and Acceleration Gains

The benchmark results printed on the terminal window Figure 5 provide clear proof of the speed gains achieved by offloading the heavy pixel math to the FPGA:

- **Software Only (PS) Processing Time:** 0.004792 seconds per frame.
- **Hardware Accelerated (PL) Processing Time:** 0.001533 seconds per frame.
- **Hardware Speedup Factor:** The hardware runs exactly $3.12\times$ faster than the software.

This $3.12\times$ speed improvement shows the value of building a custom hardware engine. Because the FPGA handles multiple pixel calculations at the exact same time, it cuts down processing lag dramatically.


```

mahmmudqatmh@mahmmudqatmh-Alienware-m16-R1: ~
[sudo] password for mahmmudqatmh:
picocom v3.1

port is       : /dev/ttyACM0
flowcontrol   : none
baudrate is   : 115200
parity is     : none
databits are  : 8
stopbits are  : 1
escape is     : C-a
local echo is : no
noinit is     : no
noreset is    : no
hangup is     : no
nolock is     : no
send_cmd is   : SZ -vv
receive_cmd is : RZ -vv -E
imap is       :
omap is       :
emap is       : crclrf,delbs,
logfile is    : none
initstring    : none
exit_after is : not set
exit is       : no

Type [C-a] [C-h] to see available commands
Terminal ready

=====
PS vs PL PERFORMANCE BENCHMARKING TEST
=====

[1/2] Starting PS (Software) Processing... Done.
[2/2] Starting PL (Hardware) Processing... Done.

===== BENCHMARK REPORT =====
Image Size       : 320 x 240 pixels
PS (Software) Runtime : 0.004790 seconds
PL (Hardware) Runtime : 0.001533 seconds
PL Acceleration Factor : 3.12x FASTER than PS
=====

===== VALIDATION STATUS =====
SUCCESS: PL Hardware data stream verified as stable and uncorrupted!
=====

===== EMBEDDED VISION REPORT =====
[ALERT]: Static Obstacle Blocked Path Segment Detected!
Obstacle Coordinates : Bounding Box Location X:[79 to 241], Y:[40 to 200]
Object Size Metric   : Dimensions 162x160 Pixels
Calculated Vector    : Center Target Axis Found at [160, 120]
=====

id 'e0b24eae-53'

ard -MMD -MP -specs=/home/mahmmudqa
atform/export/zedboard_sobel_platfor

```

Figure 5: Active Targeting Crosshair Test

5.2.2 Object Tracking and Intelligent Decision Making

Besides simply processing frames faster, the terminal log shows how the processor uses these time savings to make smart, high-level tracking choices. The moment the custom hardware finishes tracing the outlines of an object, the processor instantly reads those coordinates to calculate tracking details in real time:

- **System Alert Triggered:** The system immediately detected an object blocking its path and printed: `[ALERT]: Static Obstacle Blocked Path Segment Detected!.`
- **Target Tracking Box:** The code isolated the exact edges of the obstacle, identifying its horizontal boundaries from pixel 79 to 241 and vertical boundaries from pixel 40 to 200 ($X : [79 \text{ to } 241], Y : [40 \text{ to } 200]$).
- **Object Size Calculation:** The system calculated the absolute size of the obstacle as a

localized 162×160 pixel area.

- **Center Coordinate Extraction:** The processor found the exact middle point of the target, mapping its center axis precisely at coordinate point $[160, 120]$.

This clear division of labor proves the benefit of our hardware/software co-design. By leaving the heavy, repetitive pixel calculations to the custom FPGA hardware, the main processor is completely freed up to handle intelligent decision-making, object tracking, and obstacle avoidance maneuvers.

6 Conclusion

In conclusion, this project successfully demonstrated the complete process of designing, integrating, and testing a hardware accelerated vision pipeline on the Xilinx Zynq-7000 ZedBoard platform. By moving the complex mathematical calculations of a Sobel edge detection filter out of software and into custom FPGA hardware logic, we eliminated a major processing bottleneck for the main CPU. The live physical tests proved the immense benefit of this approach, delivering a $3.12\times$ processing speedup over running software alone. This dramatic increase in speed represents the core idea of hardware acceleration: by offloading heavy, repetitive pixel math to dedicated hardware, the main processor is fully freed to focus on intelligent, real-world tasks like target tracking, coordinate extraction, and automated obstacle avoidance.